

ORACLE®

A Cobbler's Child

Guy L. Steele Jr.
Software Architect, Oracle Labs

Code Mesh Invited Keynote
Wednesday, November 8, 2017

Copyright © 2017 Oracle and/or its affiliates (“Oracle”). All rights are reserved by Oracle except as expressly stated as follows. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted, provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers, or to redistribute to lists, requires prior specific written permission of Oracle.

The most popular programming language in computer science

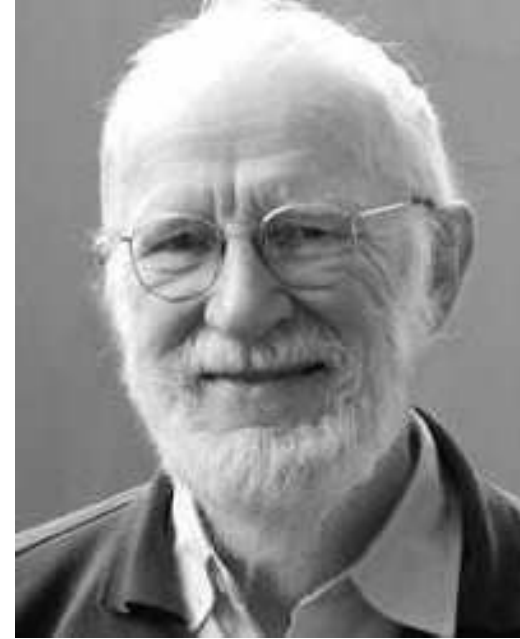
Some Early Contributors



Gerhard
Gentzen



John
Backus



Peter
Naur



Alonzo
Church

Computer Science Metanotation (CSM)

- Built-in datatypes: `boolean`, `integer`, `real`, `complex`, `sets`, `lists`, `arrays`
- Primitive expressions: `logic` and `mathematics`
- User-declared datatypes: `record` / `abstract data type` / `symbolic expression`
(**BNF** = **Backus-Naur Form**)
- Code: `Inference rules` (**Gentzen notation**)
- Conditionals: `rule dispatch` via `nondeterministic pattern-matching`
- Repetition: **overlines** and/or **ellipsis** notations, and sometimes **iterators**
- Capture-free **substitution** within a symbolic expression (**Church**)

A beautiful language—and it's getting messed up!

Example of CSM Data Declarations (BNF)

Expressions:

e	$::=$	x	Variable
		$\lambda x : \tau. e$	Abstraction
		$e_1 \ e_2$	Application
		$\Lambda \alpha : \kappa. e$	Type abstraction
		$e \ \tau$	Type application

Types:

τ, σ, ψ, v	$::=$	x	Type variable
		$\tau_1 \rightarrow \tau_2$	Function type
		$\forall \alpha : \kappa. \tau$	Polymorphic type
		$\tau_1 \ \tau_2$	Application
		$F(\overline{\tau})$	Saturated type family

κ	$::=$	$\star \mid \kappa_1 \rightarrow \kappa_2$	Kind
----------	-------	--	------

Φ	$::=$	$[\overline{\alpha : \kappa}]. F(\overline{\rho}) \sim \sigma$	Axiom equation
--------	-------	--	----------------

Adapted from Eisenberg, Vytiniotis, Peyton Jones, and Weirich,
Closed Type Families with Overlapping Equations, ACM POPL 2014, Figure 2

Example of CSM Code (Nondeterministic?) (1 of 2)

$\text{no_conflict}(\Psi, i, \bar{\tau}, j)$

Check for equation conflicts

$$\frac{\Psi = \overline{[\bar{\alpha}:\bar{\kappa}]. F(\bar{\rho}) \sim v} \quad \text{apart}\left(\overline{\bar{\rho}_j}, \overline{\rho_i[\bar{\tau}/\alpha_i]}\right)}{\text{no_conflict}(\Psi, i, \bar{\tau}, j)} \quad [\text{NC_APART}]$$

$$\frac{\text{compat}(\Psi[i], \Psi[j])}{\text{no_conflict}(\Psi, i, \bar{\tau}, j)} \quad [\text{NC_COMPATIBLE}]$$

Adapted from Eisenberg, Vytiniotis, Peyton Jones, and Weirich,
Closed Type Families with Overlapping Equations, ACM POPL 2014, Figure 4

Example of CSM Code (Nondeterministic?) (2 of 2)

$\text{no_conflict}(\Psi, i, \overline{\tau}, j)$

Check for equation conflicts

$$\frac{\Psi = \overline{[\alpha:\kappa]. F(\overline{\rho}) \sim v} \quad \text{apart}(\overline{\overline{\rho_j}, \rho_i[\tau/\alpha_i]})}{\text{no_conflict}(\Psi, i, \overline{\tau}, j)} \quad [\text{NC_APART}]$$
$$\frac{\text{compat}(\Psi[i], \Psi[j])}{\text{no_conflict}(\Psi, i, \overline{\tau}, j)} \quad [\text{NC_COMPATIBLE}]$$

Adapted from Eisenberg, Vytiniotis, Peyton Jones, and Weirich,
Closed Type Families with Overlapping Equations, ACM POPL 2014, Figure 4

Another Example of CSM Code (Deterministic?) (1 of 3)

$$\frac{}{\Gamma \vdash x_i : \tau_i} \quad \frac{\Gamma, x : \sigma \vdash M : \tau}{\Gamma \vdash \lambda x : \sigma. M : \sigma \rightarrow \tau} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau}$$

$$\frac{\Gamma \vdash M_i : \tau \quad (i = 1, \dots, \text{ar}(\text{op}))}{\Gamma \vdash \text{op}(M_1, \dots, M_{\text{ar}(\text{op})}) : \tau}$$

$$\frac{\Gamma \vdash M : \tau \times \sigma}{\Gamma \vdash \text{fst}(M) : \tau} \quad \frac{\Gamma \vdash M : \tau \times \sigma}{\Gamma \vdash \text{snd}(M) : \sigma} \quad \frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash \langle M, N \rangle : \tau \times \sigma}$$

Adapted from Muroya, Hoshino, and Hasuo,
Memoryful Geometry of Interaction II, ACM POPL 2016, Figure 1

Another Example of CSM Code (Deterministic?) (2 of 3)

$$\begin{array}{c}
 \hline
 \Gamma \vdash x_i : \tau_i \\
 \hline
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma, x : \sigma \vdash M : \tau \\
 \hline
 \Gamma \vdash \lambda x : \sigma. M : \sigma \rightarrow \tau \\
 \text{input} \quad \text{output}
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma \\
 \hline
 \Gamma \vdash MN : \tau
 \end{array}$$

$$\begin{array}{c}
 \Gamma \vdash M_i : \tau \quad (i = 1, \dots, \text{ar}(\text{op})) \\
 \hline
 \Gamma \vdash \text{op}(M_1, \dots, M_{\text{ar}(\text{op})}) : \tau
 \end{array}$$

$$\begin{array}{c}
 \Gamma \vdash M : \tau \times \sigma \\
 \hline
 \Gamma \vdash \text{fst}(M) : \tau
 \end{array}$$

$$\begin{array}{c}
 \Gamma \vdash M : \tau \times \sigma \\
 \hline
 \Gamma \vdash \text{snd}(M) : \sigma
 \end{array}$$

$$\begin{array}{c}
 \Gamma \vdash M : \tau \quad \Gamma \vdash N : \sigma \\
 \hline
 \Gamma \vdash \langle M, N \rangle : \tau \times \sigma
 \end{array}$$

Adapted from Muroya, Hoshino, and Hasuo,
Memoryful Geometry of Interaction II, ACM POPL 2016, Figure 1

Another Example of CSM Code (Deterministic?) (3 of 3)

$$\begin{array}{c}
 \hline
 \Gamma \vdash x_i : \tau_i
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma, x : \sigma \vdash M : \tau \\
 \hline
 \Gamma \vdash \lambda x : \sigma. M : \sigma \rightarrow \tau
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma \\
 \hline
 \Gamma \vdash MN : \tau
 \end{array}$$

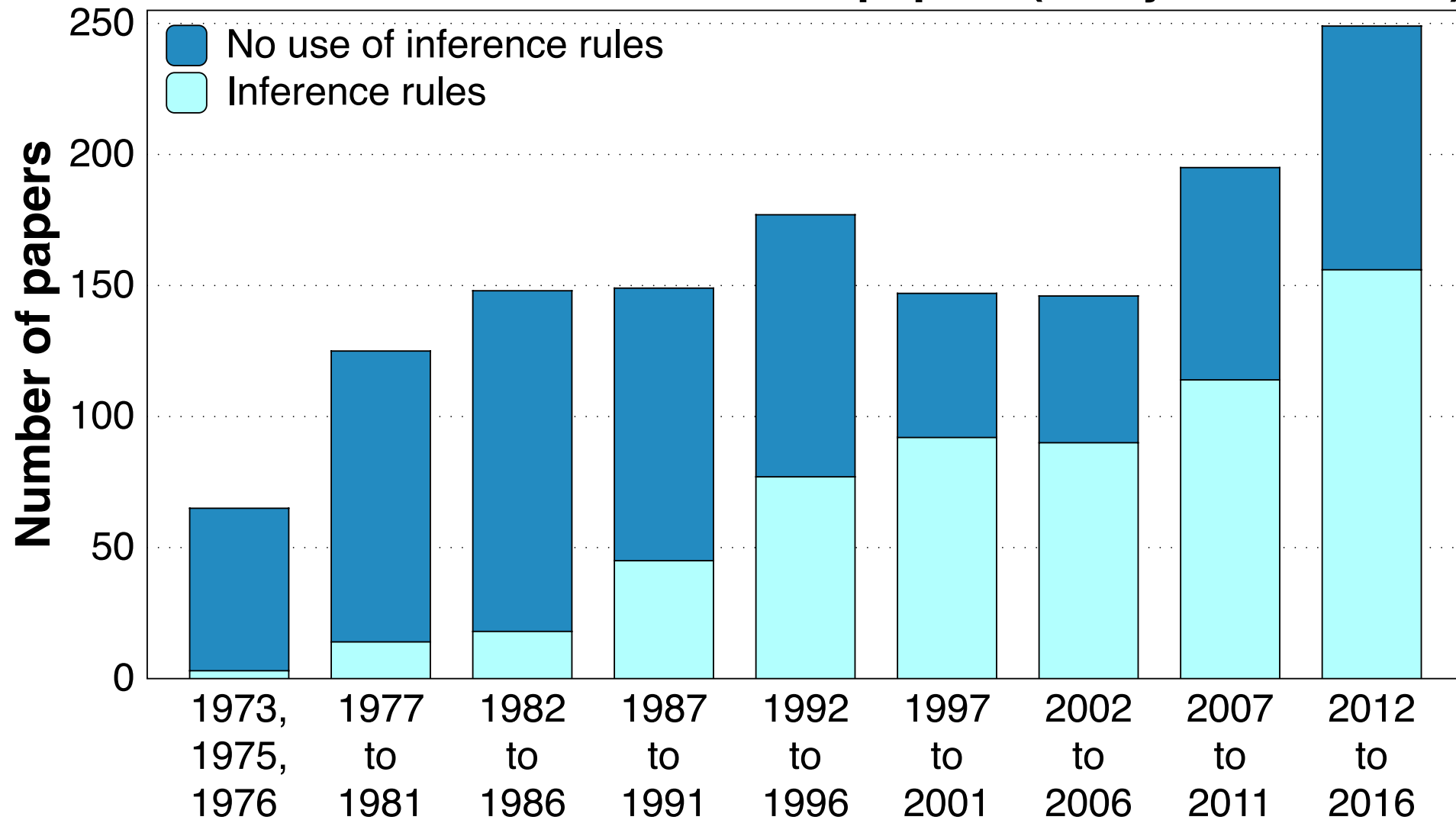
$$\begin{array}{c}
 \text{iterator} \\
 \Gamma \vdash M_i : \tau \quad (i = 1, \dots, \text{ar}(\text{op})) \\
 \hline
 \Gamma \vdash \text{op}(M_1, \dots, M_{\text{ar}(\text{op})}) : \tau \\
 \text{sequence}
 \end{array}$$

$$\begin{array}{c}
 \Gamma \vdash M : \tau \times \sigma \\
 \hline
 \Gamma \vdash \text{fst}(M) : \tau
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma \vdash M : \tau \times \sigma \\
 \hline
 \Gamma \vdash \text{snd}(M) : \sigma
 \end{array}
 \quad
 \begin{array}{c}
 \Gamma \vdash M : \tau \quad \Gamma \vdash N : \sigma \\
 \hline
 \Gamma \vdash \langle M, N \rangle : \tau \times \sigma
 \end{array}$$

Adapted from Muroya, Hoshino, and Hasuo,
Memoryful Geometry of Interaction II, ACM POPL 2016, Figure 1

Popularity of Computer Science Metanotation (1 of 2)

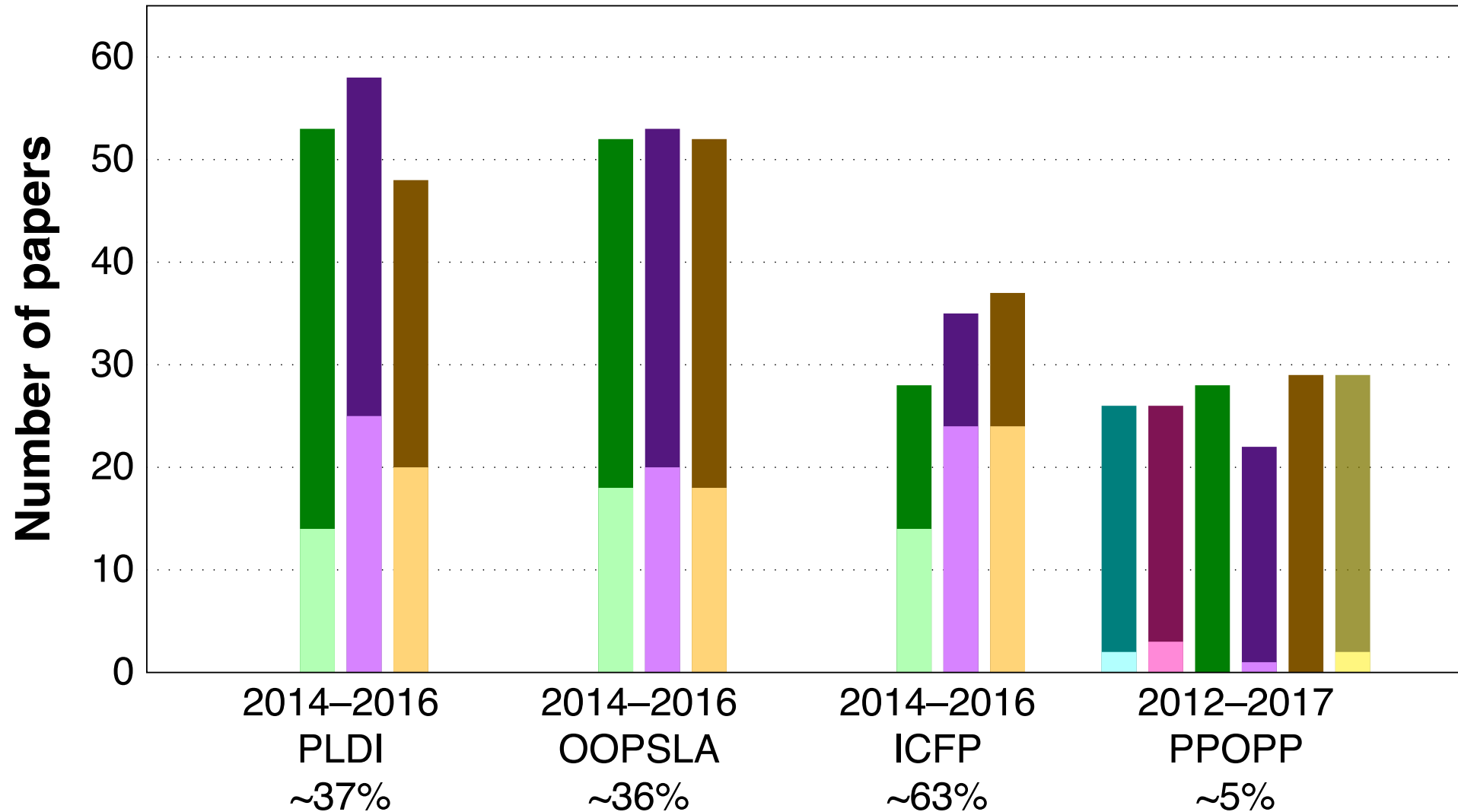
Use of inference rules in POPL papers (five-year intervals)



Analysis of 43 years of POPL conferences (1,401 papers / 17,160 pages)

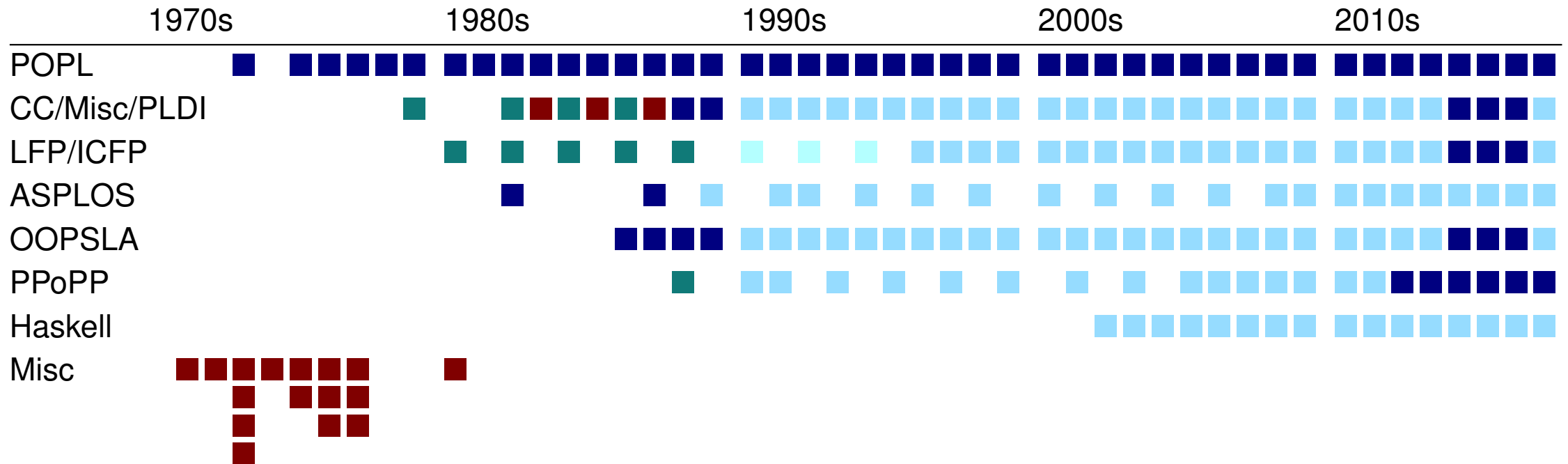
Popularity of Computer Science Metanotation (2 of 2)

Use of inference rules: other recent SIGPLAN conferences



Analysis of 3 years of PLDI, OOPSLA, and ICFP, and 6 years of PPOPP (567 papers / 8,012 pages)

SIGPLAN Conferences Researched So Far



That's 96 conferences examined out of 219.

Structure of This Talk

- Examine history and variety of five aspects of the notation:
 - Inference rules
 - BNF
 - Substitution
 - Overline
 - Ellipsis
- Identify problems that have arisen with the last three

INFERENCE RULES

Gentzen Notation (Natural Deduction)

1935 Gerhard Gentzen creates a rule notation for *natural deduction*:

Untersuchungen über das logische Schließen*). I.

Von

Gerhard Gentzen in Göttingen.

3. 1. Eine *Schlußfigur* läßt sich in der Form schreiben:

$$\frac{\mathcal{A}_1 \dots \mathcal{A}_v}{\mathcal{B}} \quad (v \geq 1),$$

wobei $\mathcal{A}_1, \dots, \mathcal{A}_v, \mathcal{B}$ Formeln sind. $\mathcal{A}_1, \dots, \mathcal{A}_v$ heißen dann die *Oberformeln*, $\mathcal{B}$ heißt die *Unterformel* der *Schlußfigur*.

$\frac{\mathcal{A} \quad \mathcal{B}}{\mathcal{A} \& \mathcal{B}}$	$\frac{\mathcal{A} \& \mathcal{B}}{\mathcal{A}}$	$\frac{\mathcal{A} \& \mathcal{B}}{\mathcal{B}}$	$\frac{\mathcal{A}}{\mathcal{A} \vee \mathcal{B}}$	$\frac{\mathcal{B}}{\mathcal{A} \vee \mathcal{B}}$
--	--	--	--	--

Gerhard Gentzen. Untersuchungen über das logische Schließen I.
Mathematische Zeitschrift 39, 1 (1935), 176–210.

Today's Computer Science Inference Rule Notation

$$\frac{\begin{array}{ccc} \textit{premise} & \textit{premise} & \textit{premise} \\ & \textit{premise} & \textit{premise} \end{array}}{\textit{conclusion}} \quad [\text{OPTIONAL LABEL}]$$

Wide variations in labels:

- Placement: left, right, upper left, upper center, lower right, ...
- Separation: adjacent to rule, or against the margin?
- Capitalization: lowercase, title caps, all caps, small caps, caps + small caps
- Mathematical symbols, or just alphanumeric?
- Size and style: normalsize, small, footnotesize; roman, italic, boldface
- Word separator: space, hyphen, period, CamelCase
- Enclosers: parentheses, brackets, none

Not really a problem!

BNF

BNF: Historical Background on Grammars

- 6th–4th century BCE** Pāṇini writes the *Aṣṭādhyāyī*, a Sanskrit grammar containing numerous concise, technical rules that describe Sanskrit morphology unambiguously and completely.
- 1914** Axel Thue studies string-rewriting systems defined by rewrite rules.
- 1920s** Emil Post studies “tag systems” in which symbols are repeatedly replaced by associated strings (this work is not published until **1943**).
- 1947** Andrey Markov and Emil Post independently prove that the word problem for semigroups (a problem posed by Thue) is undecidable.
- 1956** Noam Chomsky publishes “Three Models for the Description of Language,” which describes grammars with production rules and what we now call the “Chomskian hierarchy of grammars”.

History of **Regular Expressions** in One Slide

- 1951** Stephen Kleene develops regular expressions to describe McCulloch-Pitts (**1943**) nerve nets (uses $\vee$ for choice; considers postfix $*$, but decides to make it a *binary* operator to avoid having empty strings: “ x^*y ” means any number of copies of x , followed by y).
- 1956** Journal publication of Kleene’s technical report: binary $*$ only.
- 1958** Copi, Elgot, and Wright formulate REs using $\cdot$ and $\vee$ and postfix $*$.
- 1962** Janusz Brzozowski uses binary $+$ for $\vee$ and introduces postfix $^+$.
- 1968** AED RWORD system (Johnson et al.) uses $/$ and $\cup$ and postfix $*$.
- 1968** Ken Thompson’s paper “Regular Expression Search Algorithm” uses $|$.
- 1973** Thompson creates grep from ed editor for use by Doug McIlroy.
- 1975** Alfred Aho creates egrep (includes $()$, $|$, $*$, $+$, $?$).
- 1978** CMU Alphard project uses regular expressions with $*$, $+$, and $\#$.
- 1981** CMU FEG and IDL systems use regular expressions with $*$, $+$, and $?$.

Pretty much unchanged since 1981!

Development of **BNF**: Perlis and Samelson

1958 Alan Perlis and Klaus Samelson report on the International Algebraic Language, including “forms” for various language features.

4. *Functions F*

represent single numbers (function values), which result through the application of given sets of rules to fixed sets of parameters.

Form: $F \sim I (P, P, \dots, P)$

5. *Arithmetic expressions E* are defined as follows:

a. A number, a variable (other than Boolean), or a function is an expression.

Form: $E \sim N \quad \sim V \quad \sim F$

b. If E_1 and E_2 are expressions, the first symbols of which are neither “+” nor “−”, then the following are expressions:

$E \sim + E_1$	$\sim E_1 \times E_2$
$\sim - E_2$	$\sim E_1 / E_2$
$\sim E_1 + E_2$	$\sim E_1 \uparrow E_2 \downarrow$
$\sim E_1 - E_2$	$\sim (E_1)$

A. J. Perlis and K. Samelson. Preliminary report: International Algebraic Language.
CACM 1, 12 (December 1958), 8–22.

Development of **BNF**: Backus

1959 John Backus, influenced by “Post productions” of Emil Post, uses a specific syntax to write production rules for a context-free grammar for the International Algorithmic Language.

A single production may contain multiple alternatives.

```
<digit> ::= 0 or 1 or 2 or 3 or 4 or 5 or 6 or 7 or 8 or 9  
<integer> ::= <digit> or <integer><digit>
```

J. W. Backus. *The Syntax and Semantics of the Proposed International Algebraic Language of the Zurich ACM-GAMM Conference*. International Business Machines Corp., New York, page 14.

Development of **BNF**: Naur

1960 The “Report on Algol 60,” edited by Peter Naur, appears in CACM. It uses a slightly prettier (and easier to typeset) variant of the Backus notation. Naur introduces use of $::=$ and $|$, and makes names of nonterminals identical to equivalent English phrases used in the text.

```
⟨unsigned integer⟩ ::= ⟨digit⟩|⟨unsigned integer⟩⟨digit⟩  
⟨integer⟩ ::= ⟨unsigned integer⟩|+⟨unsigned integer⟩|  
              −⟨unsigned integer⟩
```

An Alternative: COBOL Metanotation

1960 COBOL report uses a 2-D notation. Choices are stacked vertically within braces, brackets indicate optional items, and ellipsis indicates repetition of the preceding item. *The uses of braces and brackets are documented, but the use of the ellipsis is taken for granted.*

SUBTRACT

FUNCTION: To subtract one or a sum of quantities from a specified quantity and store the result in the last named field or the specified one.

SUBTRACT {literal-1
field-name-1} [, {literal-2
field-name-2} . . .] FROM {literal-n
field-name-n}
[GIVING field-name-m] [UNROUNDED]
[; ON SIZE ERROR any imperative statement]

A Synthesis: PL/I Metanotation

1965 IBM's PL/I specification combines BNF with COBOL metanotation.

```
sum ::=          negation | sum1
sum1 ::=         product | {sum1 +
                           product} | {sum1 -
                           product}
```

An ellipsis indicates a *nonzero* number of repetitions of the preceding item;
“**[item]** ...” indicates zero or more (not “**[item ...]**”).

```
DECLARE [level] name [attribute] ...
[, [level] name [attribute] ...] ...;
```

```
(element [, element] ... { variable
                           pseudo-variable } = specification
                           [, specification] ...)
```

A specification has the following format:

```
expression-1 [ TO expression-2 [BY expression-3]
              [BY expression-3 [TO expression-2] ] [WHILE (expression-4) ]
```

IBM Operating System/360: PL/I: Language Specifications. C28-6571-1 (July 1965), pages 37, 39, and 82.

Parameterized BNF

1965 Niklaus Wirth's PL360 description used a *parameterized* form of BNF:

If in the denotations of constituents of the rule the script letters $\mathcal{R}$, $\mathcal{K}$, or $\mathcal{I}$ occur more than once, they must be replaced consistently, or possibly according to further rules given in the accompanying text. As an example, the syntactic rule

$$\langle \mathcal{K} \text{ register} \rangle ::= (\mathcal{K} \text{ register identifier})$$

is an abbreviation for the set of rules:

$$\langle \text{long real register} \rangle ::= \langle \text{long real register identifier} \rangle$$
$$\langle \text{integer register} \rangle ::= \langle \text{integer register identifier} \rangle$$
$$\langle \text{real register} \rangle ::= \langle \text{real register identifier} \rangle$$

1968 Adriaan van Wijngaarden et al. describe Algol 68 using a two-level grammar: one grammar has an infinite set of productions, which are generated by another grammar.

Niklaus Wirth. *PL360, a Programming Language for the 360 Computers*.
Stanford Computer Science Technical Report CS-TR-65-33, June 1965.

Later published in *J. ACM* 15, 1 (January 1968), 37–74.

A. Van Wijngaarden, B. J. Mailloux, J. E. L. Peck, and C. H. A. Koster.

Draft Report on the Algorithmic Language ALGOL 68. Supplement to ALGOL Bulletin 26 (March 1968), 1–84.

BLISS

1970 The BLISS language (William Wulf et al.) is described using BNF, but with a right-arrow instead of “::=”. This notation is *taken for granted*.

```
block → begin declarations compoundexpression end  
declarations → |declaration;|declarations; declaration;  
compoundexpression → |e| e; compoundexpression  
  
begin → BEGIN  
  
end → END
```

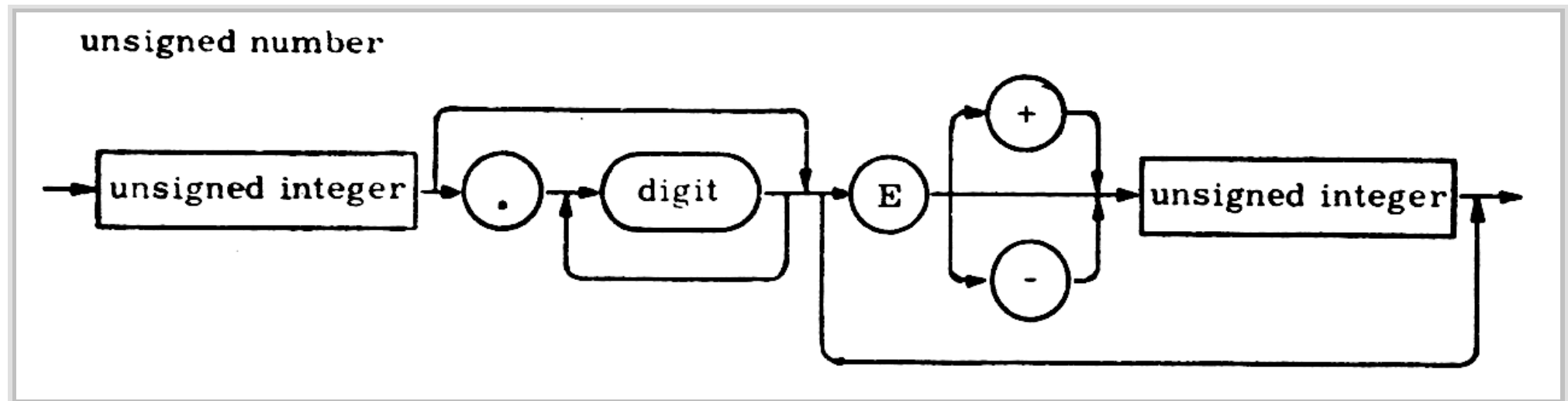
1980 The DEC BLISS documentation uses PL/I-style syntax descriptions.

W. A. Wulf, D. Russell, A. N. Habermann, C. Geschke, J. Apperson, and D. Wile.
BLISS Reference Manual: A Basic Language for Implementation of System Software for the PDP-10.
Computer Science Department, Carnegie-Mellon University (January 15, 1970), page 1.2.
Digital Equipment Corporation. *BLISS Language Guide*, Second Edition, AA-H275B-TK (January 1980).

Syntax Charts (Railway Diagrams)

1972 Burroughs CANDE language manual uses syntax charts only.

1974 PASCAL book uses both syntax charts and ALGOL 60–style BNF.



1976 Draft of FORTRAN 77 standard uses syntax charts plus PL/I-style BNF.

1979 The RED language (GREEN became Ada) uses syntax charts only.

Burroughs Corporation. *Burroughs B 6700 / C 7700 Command and Edit (CANDE) Language Information Manual*. 5000318 (2 October 1972).

Kathleen Jensen and Niklaus Wirth. *PASCAL User Manual and Report*. Springer-Verlag (1974), page 116.

Draft proposed ANS FORTRAN BSR X3.9 X3J3/76. *SIGPLAN Notices* 11, 3 (March 1976), 1–212.

John Nestor and Mary Van Deusen. *RED Language Reference Manual*. IR-310-2, Intermetrics (8 March 1979).

Wirth Syntax Notation (WSN)

1977 Niklaus Wirth publishes “What can we do about the unnecessary diversity of notation for syntactic definitions?” in CACM, solving the problem of having too many BNF variants by proposing yet another. It catches on.

```
syntax      = {production}.
production  = identifier "=" expression ".".
expression  = term {"|" term}.
term        = factor {factor}.
factor      = identifier | literal | "(" expression ")" |
               "[" expression "]" | "{" expression "}".
literal     = " " " " " " character {character} " " " " " ".
```

Repetition is denoted by curly brackets, i.e. {a} stands for ϵ | a | aa | aaa | Optionality is expressed by square brackets, i.e. [a] stands for a | ϵ . Parentheses merely serve for grouping, e.g. (a|b)c stands for ac | bc.

1996 ISO/IEC Standard 14977:1996 *Extended BNF* (very similar to WSN).

Other BNF Variants

- 1976** Stanford's SAIL language uses BNF with repeated “`::=`” and no “`|`”.
- 1978** CMU Alphard project uses regular expressions *in BNF* with `*`, `+`, and `#`.
- 1980** Ada specification uses BNF, but with “**is**” for “`::=`” and “**or**” for “`|`”.
- 1981** CMU FEG and IDL use regular expressions *in BNF* with `*`, `+`, and `?`.
- 1984** *C: A Reference Manual* (Harbison and Steele) uses REs in BNF.
- 1984** *Common Lisp: The Language* (Steele et al.) uses REs in BNF.
- 1995** *Python Reference Manual* (Release 1.2) uses `*` and `+` in BNF, but brackets (rather than `?`) for optional items.
- 1998** Haskell 98 Report uses BNF, with `->` for `::=`, and also uses ellipsis.
- 1998** *Ruby Language Reference Manual* (1.4.6) uses `*` and `+` in “pseudo BNF” (somewhat like WSN), but brackets (rather than `?`) for optional items.

C-style BNF

1978 Brian Kernighan and Dennis Ritchie publish *The C Programming Language*, which uses yet another format for grammar rules.

iteration-statement:

while (*expression*) *statement*

do *statement* **while** (*expression*);

for (*expression*_{opt}; *expression*_{opt}; *expression*_{opt}) *statement*

assignment-operator: one of

= *= /= %= += -= <<= >>= &= ^= |=

1985 *The C++ Programming Language* (Bjarne Stroustrup) uses C-style BNF.

1996 *The Java Language Specification* (Gosling et al.) uses C-style BNF.

2000 *C# Language Specification* (Hejlsberg et al.) uses C-style BNF.

2012 *The F# 2.0 Language Specification* (Don Syme) uses C-style BNF but with special treatment of ellipsis (curiously defined as postfix).

**We have seen a huge variety
of BNF variations
in the last six decades.**

It hasn't been a problem.

Example of CSM Data Declarations [Again]

Expressions:

e	$::=$	x	Variable
		$\lambda x : \tau. e$	Abstraction
		$e_1 \ e_2$	Application
		$\Lambda \alpha : \kappa. e$	Type abstraction
		$e \ \tau$	Type application

Types:

τ, σ, ψ, v	$::=$	x	Type variable
		$\tau_1 \rightarrow \tau_2$	Function type
		$\forall \alpha : \kappa. \tau$	Polymorphic type
		$\tau_1 \ \tau_2$	Application
		$F(\overline{\tau})$	Saturated type family

κ	$::=$	$\star \mid \kappa_1 \rightarrow \kappa_2$	Kind
----------	-------	--	------

Φ	$::=$	$[\overline{\alpha:\kappa}]. F(\overline{\rho}) \sim \sigma$	Axiom equation
--------	-------	--	----------------

Adapted from Eisenberg, Vytiniotis, Peyton Jones, and Weirich,
Closed Type Families with Overlapping Equations, ACM POPL 2014, Figure 2

Example of CSM Data Declarations [Again]

Expressions:

e	$::=$	x	Variable
		$\lambda x : \tau. e$	Abstraction
		$e_1 \ e_2$	Application
		$\Lambda \alpha : \kappa. e$	Type abstraction
		$e \ \tau$	Type application

Types:

τ, σ, ψ, v	$::=$	x	Type variable
		$\tau_1 \rightarrow \tau_2$	Function type
		$\forall \alpha : \kappa. \tau$	Polymorphic type
		$\tau_1 \ \tau_2$	Application
		$F(\overline{\tau})$	Saturated type family
κ	$::=$	$\star \mid \kappa_1 \rightarrow \kappa_2$	Kind
Φ	$::=$	$[\overline{\alpha:\kappa}]. F(\overline{\rho}) \sim \sigma$	Axiom equation

Adapted from Eisenberg, Vytiniotis, Peyton Jones, and Weirich,
Closed Type Families with Overlapping Equations, ACM POPL 2014, Figure 2

The “Consistent Substitution” Convention

If we took the definition of BNF literally—every nonterminal can be replaced by any string derived from that nonterminal—then a sentence such as

A value of type τ may be assigned to any variable of type τ .

could be expanded to

A value of type `int` may be assigned to any variable of type `bool`.

which is nonsense. Instead, we require *consistent substitution*: within a given context (*other* than the RHS of a BNF rule), if a nonterminal is mentioned more than once, the same expansion must be used for each occurrence:

A value of type `int` may be assigned to any variable of type `int`.

The “Decorated Nonterminals” Convention

If we took the definition of BNF literally—every nonterminal can be replaced by a string derived from that nonterminal—then a sentence such as

If $\tau_1 = \tau_2$, then $\tau_1 <: \tau_2$.

would be expanded (for example, with $\tau \rightarrow \text{int}$) to

If $\text{int}_1 = \text{int}_2$, then $\text{int}_1 <: \text{int}_2$.

which is nonsense. Instead, we recognize a *decorated nonterminal* as being a distinct nonterminal having the same productions as the undecorated form:

If $\text{int} = \text{bool}$, then $\text{int} <: \text{bool}$.

If $\text{int} = \text{int}$, then $\text{int} <: \text{int}$.

SUBSTITUTION

Substitution Notation

1932 Alonzo Church uses the notation $S_Y^X U$ for *substitution* in a formula:

We assume an understanding of the operation of *substituting* a given symbol or formula *for a particular occurrence* of a given symbol or formula.

And we assume also an understanding of the operation of substitution throughout a given formula, and this operation we indicate by an S , $S_Y^X U$ representing the formula which results when we operate on the formula U by replacing X by Y throughout, where Y may be any symbol or formula but X must be a single symbol, not a combination of several symbols.

Note: the variable to be substituted for is on *top*, and the replacing term is on the *bottom*!

1941 Alonzo Church publishes *The Calculi of Lambda-Conversion*.

II. To replace any part $(\lambda x M)N$ of a formula by $S_N^x M$, provided that the bound variables of M are distinct both from x and from the free variables of N .

Alonzo Church. A Set of Postulates for the Foundation of Logic.
Annals of Mathematics Second Series 33, 2 (April 1932), 346–366.

Alonzo Church. *The Calculi of Lambda-Conversion*. Princeton University Press (1941), page 12.

Nowadays we write

$$e[v/x]$$

(or something like it)

for the result of substituting v for x in e .

How many variations are there?

32 Varieties of Substitution Notation: POPL 1973–2016

e_v^x	5	$e _v^x$	6	$e _x^v$	1
$e_{\frac{v}{x}}$	1	$e[v x]$	1	$e(v/x)$	1
$[v/x]e$	67	$e[v/x]$	133	$e\{v/x\}$	25
$[v/x]e$	1	$e[v/x]$	6	$e\{v/x\}$	5
		$e[v/x]$	2	$e\{v/x\}$	4
$[x \mapsto v]e$	9	$e[x \mapsto v]$	17	$e\{x \mapsto v\}$	1
$[x := v]e$	2	$e[x := v]$ *	21		
$[x \rightarrow v]e$	1	$e[x \rightarrow v]$	2	$e\{x \rightarrow v\}$	1
$\{v/x\}e$	6	$e[x/v]$	5	$e\{[v/x]\}$	2
$\{x \mapsto v\}e$	4	$e[x \leftarrow v]$	7	$e\{\{x \leftarrow v\}\}$	1
$\llbracket v/x \rrbracket e$	2	$e[v \setminus x]$	1	$e\{x \leftarrow v\}$	4
$e _{x \rightarrow v}$	1				

Most popular during 1973–2016 are highlighted.

* Used by H. P. Barendregt in *The Lambda Calculus: Its Syntax and Semantics* (1980).

Substitution: **POPL 2012–2016** and **Others 2014–2016**

e_v^x	5		e_v^x	6		e_x^v	1
$e_{\frac{v}{x}}$	1		$e[v x]$	1		$e(v/x)$	1
$[v/x]e$	67	17	$e[v/x]$	133	37	$e\{v/x\}$	25 7
$[v/x]e$	1		$e[v/x]$	6		$e\{v/x\}$	5
$[v/x]e$		1	$e[v/x]$	2	2	$e\{v/x\}$	4
$[x \mapsto v]e$	9	2	$e[x \mapsto v]$	17	8	$e\{x \mapsto v\}$	1
$[x := v]e$	2		$e[x := v]$	21	3	$e\{x := v\}$	1
$[x \rightarrow v]e$	1		$e[x \rightarrow v]$	2	1	$e\{x \rightarrow v\}$	1
$\{v/x\}e$	6		$e[x/v]$	5	1	$e\{[v/x]\}$	2
$\{x \mapsto v\}e$	4		$e[x \leftarrow v]$	7	1	$e\{\{x \leftarrow v\}\}$	1
$\llbracket v/x \rrbracket e$	2		$e[v \setminus x]$	1		$e\{x \leftarrow v\}$	4 1
$e _{x \rightarrow v}$	1						

Of these 34, 15 were used at POPL during **2012–2016**;
 4 more were used at other SIGPLAN conferences during **2014–2016**.
 Substitution was used in over 1/3 of POPL papers during **2012–2016**.

Breaking News: Last Week I Found Five More

$e[[x \setminus v]]$	1984 Lisp and Functional Programming, p. 76
$e[x \setminus v]$	1986 Lisp and Functional Programming, p. 18
$(v/x)e$	1988 Lisp and Functional Programming, p. 158
$e([v/x])$	1989 Principles of Programming Languages, p. 13
$[x \setminus v]e$	1989 Principles of Programming Languages, p. 72

That makes 39 kinds I have found in SIGPLAN conferences (so far).

Substitution: **A Moderate Problem**

By far the most popular form is

$$e[v/x]$$

but about every once every five years we see

$$e[x/v]$$

which gets it *backwards*.

You can't count on the variable names to tip you off,
because different authors use different names.

One paper published in the last year used *both* forms.

Substitution: **A Huge Problem**

The forms $e[x \mapsto v]$ and $e[x := v]$ as well as $e[v/x]$
(and variants that are prefix and/or use braces)
are frequently used for **substitution**
(about 1/6 of all POPL papers).

But they are also widely used for another purpose:
function update (also called **map update** and **storage update**)!

$$(f[x \mapsto v])(z) = \mathbf{if} \ z = x \ \mathbf{then} \ v \ \mathbf{else} \ f(z)$$

Use of both in one paper can make it very hard to read.

And lately most authors are taking all these notations for granted.

Substitution: **My Recommendations**

- Use postfix forms (clearly more popular).
- Use either $/$ (most popular) or $\rightarrow$ (arguably clearer). *Never* use $\leftarrow$.
- If you use $/$, do *not* make names smaller (it only makes them less readable).
- Reserve $\mapsto$ for function/map update and $:=$ for storage/heap update.
- Use brackets $[]$ for operators; use braces $\{ \}$ for collections.

applications		operators		(singleton) collections
substitution	$e[v/x]$	a substitution	$\sigma = [v/x]$	a map $\Gamma = \{x \mapsto v\}$ a heap $H = \{x := v\}$
substitution	$e[x \rightarrow v]$	a substitution	$\sigma = [x \rightarrow v]$	
map update	$\Gamma[x \mapsto v]$	a map update	$u = [x \mapsto v]$	
heap update	$H[x := v]$	a heap update	$u = [x := v]$	

- Mnemonic for $e[v/x]$: “Within e , v *supersedes* (‘sits over’) x .”
- Mnemonic for $e[x \rightarrow v]$: “Within e , x *becomes* v .”

OVERLINE

Overline Notation (and Dots and Parentheses)

- 1484** Nicolas Chuquet uses an *underline* for mathematical grouping.
- 1525** Christoff Rudolff uses dots: $\sqrt{\cdot}12 + \sqrt{\cdot}140$ means $\sqrt{12 + \sqrt{140}}$.
- 1556** Niccolò Tartaglia uses parentheses () for mathematical grouping.
- 1631** William Oughtred uses double dots : to indicate grouping.
- 1637** René Descartes attaches an *overline* to $\sqrt{}$, producing $\sqrt{}$.
- 1640** Jan Stampioen uses all three *together*: “ $\sqrt{\cdot}(\overline{aaa + 6aab + 9bba})$ ”.
- 1646** Frans van Schooten, editing Vieta’s works, uses overline for grouping.
- 1702** Gottfried Leibniz begins using parentheses in preference to overline.
- 1728–** Euler and two Bernoullis popularize use of parentheses.
- 1857** Giuseppe Peano reintroduces dots: “ $a:bc.d$ ” means “ $a((bc)d)$ ”.
- 1910** Russell and Whitehead (*Principia Mathematica*) adopt Peano’s dots.

Three notations for grouping duking it out for five centuries!

Vectors

- 1813** Jean-Robert Argand graphs complex numbers, speaks of $i = \sqrt{-1}$ as a rotation in the plane, and proposes the notation $\overrightarrow{ab}$ for vectors.
- 1833** William Rowan Hamilton recasts the theory of complex numbers as an algebra on pairs of reals (a_1, a_2) .
- 1833–43** Hamilton seeks an algebra for triplets and polyplets (that is, tuples).
- 1843** Hamilton discovers the quaternions $a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$.
- 1844–46** Hamilton reformulates quaternions without $\mathbf{ijk}$ coordinates, describing a quaternion as the sum of a scalar and an (imaginary) vector.
- 1873** James Maxwell uses quaternions to describe electricity and magnetism.
- 1881** Josiah Willard Gibbs establishes $\cdot$ and $\times$ for dot and cross product.
- 1882** Oliver Heaviside advocates ditching scalars and simply using vectors.
- 1890–94** Big fight between “quaternionists” and “vectorists” in physics!

Vectors and Overlines at POPL (1 of 3)

1975–1981 Both $\vec{a}$ and $\overline{a}$ are used to denote a vector, list, sequence, or set that is *enclosed*: $\vec{a} = \langle a_1, a_2, \dots, a_m \rangle$ or $\overline{x} = \{x_1, \dots, x_k\}$.

1981 Overline notation is *taken for granted*.

1989 $\vec{X}$ indicates an *unenclosed sequence*.

So far, the **semantic model** is that an overline marks a variable as representing a sequence, and the obvious **syntactic model** is that you make copies of the overlined name and attach sequential subscripts starting from 1.

Vectors and Overlines at POPL (2 of 3)

1990 First explicit claim that the elements may be *metasyntactic variables*

1990 First use of an *implicit unit of replication*: $\overline{m} : \overline{\sigma}$ means

$m_1 : \sigma_1, \dots, m_k : \sigma_k$ rather than $m_1, \dots, m_k : \sigma_1, \dots, \sigma_k$

Problem: In a general and purely syntactic model of overline notation, just how large is the implicit unit of syntactic replication?

1993 First claim that overline may apply to *any syntactic object*:

$\overline{m : \sigma}$ means $m_1 : \sigma_1, \dots, m_k : \sigma_k$

Problem: Where should subscripts be attached?

Why is the result “ $m_1 : \sigma_1, \dots, m_k : \sigma_k$ ”

rather than “ $m : \sigma_1, \dots, m : \sigma_k$ ”

or “ $m_1 :_1 \sigma_1, \dots, m_k :_1 \sigma_k$ ” ?

(It's easy to come up with reasons; but so far no one has stated them!)

Vectors and Overlines at POPL (3 of 3)

1994 First use of *nested* overlines.

1996 Overline notation taken for granted, but first explicit statement of the “equal-length convention”: “We implicitly assume in $[\overline{z}/\overline{y}]$ that the sequence $\overline{y}$ is linear and of the same length as $\overline{z}$.”

1997 First statement of general *pointwise extension*:

“By abuse of notation, operations on singletons are implicitly extended pointwise to sequences.” But immediately we run into a problem!

The Problem (1997)

What does $\Gamma(\bar{b}) = [\bar{T}/\bar{X}]\bar{P}$ mean? Pure pointwise extension gives

$$\Gamma(b_1) = [T_1/X_1]P_1$$

and ...

$$\text{and } \Gamma(b_n) = [T_n/x_n]P_n$$

But the authors really wanted

$$\Gamma(b_1) = [T_1/X_1, \dots, T_m/x_m]P_1$$

and ...

$$\text{and } \Gamma(b_n) = [T_1/X_1, \dots, T_m/x_m]P_n$$

where b and P are of the same length n ,
 T and X are of length m (possibly different from n),
and the same substitution is used in all equations.

A Solution? Nested Overlines (1 of 2)

Some authors would instead write $\overline{\Gamma(b) = [\overline{T/X}]P}$.

Superficially, this seems natural.

But how do we know that Γ is not supposed to be subscripted?

The Essential Contradiction

In about the last 15 years, we have found that we want *both* of these usages:

We want $\overline{p = [v/x]q}$ to mean

$$p_1 = [v_1/X_1, \dots, v_m/x_m]q_1$$

and ...

$$\text{and } p_n = [v_1/X_1, \dots, v_m/x_m]q_n$$

where all the
substitutions are
the *same*

but we want $\text{case } e \text{ of } \overline{K \bar{y} \rightarrow e'}$ to mean

case e of

$$K_1 y_{11} \dots y_{1m_1} \rightarrow e'_1$$

...

$$K_n y_{n1} \dots y_{nm_n} \rightarrow e'_n$$

where each case clause may
have *different* y variables and
indeed a *different number* of
 y variables

With a purely syntactic theory, we can't have it both ways.

What Do We Want from a Formal Overline Notation?

- $\overline{str}$ can expand to any number of copies of str .
- Each copy of str may be expanded *differently*.
 - BNF nonterminals in str may be expanded differently in each copy.
 - Nested overlines in str may be expanded differently in each copy.
- *Within each copy of str , multiple occurrences of the same BNF nonterminal must be expanded in the same way (as usual).*
- If $\overline{str}$ is mentioned more than once in a given context, the expansion of each occurrence must be the *same* (just as for BNF nonterminals).
- If a variable v occurs within str , copy i of the str must refer to v_i .
- All variables occurring in str must have the *same length*.

What Principles Do We See?

- If a notation explicitly appears in more than one place on the paper, all such explicit copies must be given the *same* expansion.
 - $f(e, e, e)$ means all three arguments must be the *same* expression e .
 - One can override this default by decorating nonterminals: $f(e_1, e_2, e_3)$
- But if “virtual copies” are made by the overline notation, the copies may have *different* expansions.
 - $f(\overline{e})$ means the arguments to f may be *different* expressions.
 - And we have a way to express this: decorating nonterminals!
For example, $\overline{x}$ when expanded to three copies means x_1, x_2, x_3 .
If x is a nonterminal, then x_1, x_2 , and x_3 may have different expansions.
And if x is a variable, then copy i does indeed refer to x_i .
 - But we have no way to override this default! **This is the problem.**

Solving the **Essential Contradiction**

We propose to borrow an idea from *quasiquoting*:

- ‘(lambda ,vars ,body) means “make a copy of the S-expression (lambda ,vars ,body), but a comma means ‘except here’: the *value* of the expression following the comma should be used”.

Clojure has this, too: ‘(lambda ~vars ~body).

A similar idea was used for parallelism in Connection Machine Lisp (**1986**):

- $\alpha(+ (* 9/5 \bullet \text{temps}) 32)$ means “evaluate many copies of the expression $(+ (* 9/5 \bullet \text{temps}) 32)$, but a bullet means ‘except here’: the value of the expression following the bullet is a *vector*, so please use a different vector element in each copy”.

Guy L. Steele Jr. and W. Daniel Hillis.

Connection Machine Lisp: Fine-grained Parallel Symbolic Processing.
Proc. 1986 ACM Conference on LISP and Functional Programming, 279–297.

Adding **Underlines** to Overlines

We propose this modification to overline notation in CSM:

- $\overline{str}$ can expand to any number of copies of str , and each copy of str may be expanded *differently*, but an underline means “except here”: underlined portions of str must be expanded the *same* way in each copy.

Therefore for our examples we can write:

$$\overline{p = \underline{[v/x]}q}$$

same substitution in each outer copy

$$\text{case } e \text{ of } \overline{K \overline{y} \rightarrow e'}$$

as before

$$\overline{\underline{\Gamma} \vdash x : \tau}$$

same Γ in each copy

$$\overline{\underline{\Gamma}(b) = \underline{[T/X]}P}$$

same Γ and same substitution in each outer copy

The dimensionality of each variable is simply the number of overlines minus the number of underlines.

Mechanics for a Formal Overline Notation

A formal theory of overline expansion must track two kinds of constraints:

- Requirements for identical expansion.
- Requirements that variables be the same length.

Insight: If we use the original “subscript attachment” model for overlines, but allow underlines to override automatic subscript attachment, then the usual rules for BNF nonterminals will enforce precisely the desired same-expansion constraints. (Length constraints must still be tracked separately.)

Various constraints and traditions suggest that:

- Overlines should be expanded *before* BNF nonterminals.
- Overlines should be *outermost first*.
- Substitutions should be expanded *after* BNF nonterminals.

A Simple (?) Formal Model for Overline Expansion

To expand an outermost overline:

- Freely choose an integer length n .
- Replaced the overlined string with n copies of the string.
- In copy k ($1 \leq k \leq n$), for every (possibly already decorated) variable or BNF nonterminal that is *not underlined*, attach k as a subscript.
 - Record the fact that all items to which subscripts are attached are constrained to have the same length.
- In each copy, from any underlined material remove just one underline.
- Now perform expansions in the replacement material.

A Simple (!) Formal Model for Context Expansion

To expand an entire context (inference rule, right-hand side of a BNF rule, or text sentence or paragraph):

- Repeatedly expand outermost overlines until none are left.
- Expand all BNF nonterminals, obeying the same-expansion and decorated-nonterminal rules.
- Expand all substitution notations.

The expansion of an entire context is valid only if the various “free choices” for overline lengths have been made so that all length constraints are satisfied.

What about Cases Simple Overlines Can't Handle?

There are already solutions in the literature.

One can explicitly notate bindings of index variables and use them as explicit subscripts:

$$\overline{\Gamma(b_i) = [\overline{T_j/X_j}^j] P_i}^i$$

and if necessary explicitly specify ranges for these bound variables:

$$\overline{\Gamma(b_i) = [\overline{T_j/X_j}^{1 \leq j \leq m}] P_i}^{0 \leq i < n}$$

I endorse such explicit-binding overline notations for difficult cases.

ELLIPSIS

The **Ellipsis** (Dot Dot Dot)

Most readers will have encountered the *dotdotdot* notation already. It is a notation that is rarely introduced properly; mostly, it is just used without explanation as in, for example, ‘ $1 + 2 + \dots + 20 = 210$ ’

—Roland Backhouse, *Program Construction* (Wiley, 2003), p. 137

In the Past, We Have Used Ellipsis to Explain Overline

“ $\overline{x}$ ” means “ $x_1, \dots, x_n$ ”

But what does “ $x_1, \dots, x_n$ ” mean?

(Or “ $x_1, x_2, \dots$ ”? Or “ $e_1, \dots, e_i, \dots, e_n$ ”?)

We propose to explain the *ellipsis* notation
by providing a formal transformation to *overline* notation
(whose formal definition need not rely on ellipses).

The Basic Idea

- Predefine a set of standard *ellipsis usage patterns* to be supported.
- For each use of ellipsis, expansion must identify a matching usage pattern.
- Each pattern includes (a) one or more ellipses, (b) some number of copies of a *separator string*, and (c) *matchable strings*.
- Use unification-like matching on the matchable strings to find a *common structure* parameterized by one variable (an integer index) and a set of unifying *substitutions* for that variable.
- Construct an overline notation using one copy of the common structure and one copy of the separator string, and use the substitution expressions to specify the range and/or verify constraints.

Example

$$x_0, \dots, x_{n-1}$$

The separator is “,”.

The matchable strings are x_0 and x_{n-1} .

The common structure is x_i
with substitutions $[0/i]$ and $[n-1/i]$.

The result is $\overline{x_i}^{0 \leq i \leq n-1}$.

Conclusions

- Computer Science Metanotation is a symbolic programming language with its own distinctive syntax, semantics, and idioms.
- CSM should be an explicit object of study in our community.
- CSM is a living language and has changed over the last four decades (and some of its notational ideas go back centuries).
- We now have problems with substitution and overlines. These can be fixed.
- We should develop a complete formal theory of the language, including **overline notation** and **ellipsis notation** (including nested cases) and their interaction with BNF and substitution. I have made a start.
- We should apply the techniques developed for other languages to CSM to build interpreters, compilers, IDEs, correctness checkers, and other tools.
- There are interesting opportunities for parallel execution of CSM and the use of parallel algorithms in associated tools.

Questions?

Comments?

ORACLE®